This plan specifies a full-duplex, interruptible, voice-enabled AI agent that runs on a Raspberry Pi. It installs an MCP-capable agent harness (Pipecat 1.13.0), builds two local MCP servers wrapping SOTA ASR (faster-whisper 1.2.1) and TTS (Piper 1.8.0), installs a production VAD (Silero VAD 6.2.3), and runs the agent against real-world tasks. Commercial APIs may supply the LLM brain, but the ASR/TTS/VAD MCP servers are built and run on the Pi. A standardized latency benchmark quantifies voice-interaction quality. Work is split across three agents with explicit ownership and verification gates.

Voice Duplex â€” Full-Duplex Voice AI Agent on Raspberry Pi

An agent-oriented engineering plan: every step, microstep, owner, and gate

AI 231 Â· Dumalaog Â· MEng AI Notebooks

PREPARED BY	OnIt (agent) Â· coordinated with Codex (thinker/auditor) + Antigravity (implementer)
DATE	2026-10-19 (Asia/Manila)
TARGET HARDWARE	Raspberry Pi 4 (4/8 GB) or Pi 5 (4/8/16 GB) â€” 64-bit Raspberry Pi OS Lite
CLASSWORK DELIVERABLE	Standardized voice-interaction latency benchmark (Section 9)
STATUS	Planning â€” greenfield (no prior ME3 scaffolding in workspace)

EXECUTIVE SUMMARY

1 Mission, Scope & Constraints

Goal: a conversational, natural, interruptible voice agent on Pi 4/5 that listens and speaks simultaneously (full-duplex), can be barged into mid-sentence, and completes real-world tasks via MCP tool calls. The system must be reproducible, measurable, and safe to hand off between agents.

1.1 In scope

- Agent harness with MCP support (Pipecat + MCP Python SDK v2).
- Two locally-built MCP servers: ASR (faster-whisper) and TTS (Piper), running on the Pi.
- Silero VAD for voice activity detection and endpointing, exposed to the pipeline.
- Full-duplex audio: simultaneous capture + playback, barge-in, turn-taking.
- Real-world task execution through MCP tools (weather, timers, ME2 command model, etc.).
- Standardized latency benchmark (classwork) with a fixed measurement protocol.

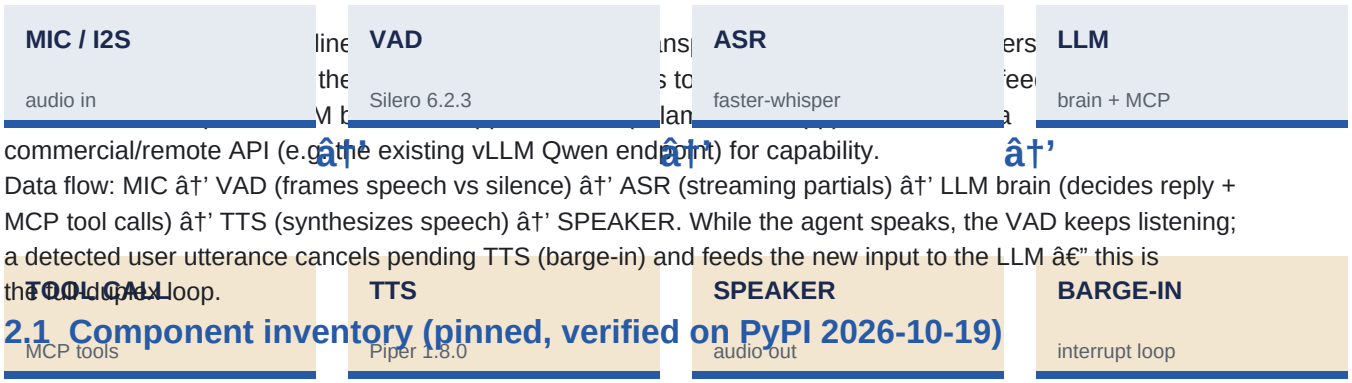
1.2 Out of scope / deferred

- Retraining ASR/TTS/VAD models (use SOTA pre-trained weights; fine-tune only if required later).
- Multi-speaker diarization, translation, and vision (multimodal) in v1.
- Cloud-hosted agent (target is on-Pi; cloud is used only for the optional LLM brain).

1.3 Hard constraints (from workspace AGENTS.md)

- Single shared environment (.venv) on the dev host; on the Pi, one dedicated venv for ME3.
- No secrets in files/logs/chat/commits; API keys in OS keychain / env, referenced by name only.
- Preserve existing Pi deployments (VCM at ~/Desktop/dandan, Kiko) â€” ME3 lives in its own folder.
- Every claim of 'ran / verified' must be backed by an actually-executed command and recorded evidence.
- Mandatory handoff update after material changes (Update-HandoffIndex.ps1 + HANDOFF_INDEX.md).

2 System Architecture



commercial/remote API (e.g. the existing vLLM Qwen endpoint) for capability.

Data flow: MIC â†’ VAD (frames speech vs silence) â†’ ASR (streaming partials) â†’ LLM brain (decides reply + MCP tool calls) â†’ TTS (synthesizes speech) â†’ SPEAKER. While the agent speaks, the VAD keeps listening; a detected user utterance cancels pending TTS (barge-in) and feeds the new input to the LLM â€” this is the full-duplex loop.

2.1 Component inventory (pinned, verified on PyPI 2026-10-19)

COMPONENT	PACKAGE	VERSION	ROLE
Harness	pipecat-ai	**1.13.0**	realtime voice pipeline, transports, MCP tool calling
MCP SDK	mcp (Python)	**2.3.0**	official Model Context Protocol, FastMCP server builde
VAD	silero-vad	**6.2.3**	voice activity detection, endpointing, ONNX/TorchScri
ASR	faster-whisper	**1.2.1**	CTranslate2 Whisper; int8, streaming, on-Pi capable
TTS	piper-tts	**1.8.0**	low-latency neural TTS, many voices, real-time on Pi
Audio I/O	sounddevice / PortAudio	**latest**	full-duplex capture + playback
LLM (opt.)	ollama / remote vLLM	**â€”**	swappable brain; local or commercial API

3 Multi-Agent Division of Labor

ONIT (planner/coordinator)

Owns this plan, task sequencing, and the AGENT_TASK_STATE.json contract. Decides scope from evidence, writes the handoff, and escalates user-only decisions. Does not silently claim Pi results.

CODEX (thinker / auditor)

Decides scope from evidence, reconciles requirements, reviews every diff, runs independent checks, and updates the handoff. The gatekeeper: no step is 'verified' until Codex re-runs the check.

ANTIGRAVITY (implementer)

Inspects before changing, implements the agreed task, and reports exact files/commands/results. Flags uncertainty instead of inferring success. Builds the MCP servers, VAD integration, and benchmark.

3.1 Collaboration protocol (every task)

â€¢ Codex writes the task spec + acceptance criteria into AGENT_TASK_STATE.json (owner, next_step).

â€¢ Antigravity implements, records exact commands + output, marks status 'implemented' (not 'verified').

â€¢ ~~Codex independently re-runs the acceptance check; only then sets status 'verified' with evidence~~ ref.

â€¢ OnIt updates HANDOFF_INDEX.md (change log, current state, open items) before the task closes.

â€¢ Limit-stall watchdog (30-min schedule) resumes a stalled owner from next_step using the shared .venv.

4 Task 1 â€” Install MCP-Capable Harness on Pi

Owner: Antigravity Â· Auditor: Codex Â· Gate: imports pass + a hello-pipeline runs end-to-end.

4.1 Pre-flight (microsteps)

â€¢ Confirm Pi model/RAM: `cat /proc/cpuinfo`, `free -m`, `uname -m` (must be arm64).

â€¢ Confirm OS: 64-bit Raspberry Pi OS Lite; `sudo apt update && sudo apt -y upgrade`.

â€¢ Install system deps: `sudo apt -y install python3-venv python3-dev portaudio19-dev libasound2-dev`.

â€¢ Create isolated folder + venv: `mkdir -p ~/me3 && cd ~/me3 && python3 -m venv .venv`.

â€¢ Activate: `source .venv/bin/activate`; pin `python -m pip install -U pip wheel setuptools`.

4.2 Harness + MCP install (microsteps)

â€¢ `pip install pipecat-ai==1.13.0 mcp==2.3.0` â€” harness + official MCP SDK.

â€¢ `pip install sounddevice numpy` â€” full-duplex audio I/O (PortAudio backend).

â€¢ Verify: `python -c "import pipecat, mcp, sounddevice; print('ok')"`.

â€¢ Smoke test: run Pipecat's minimal pipeline with a mock transport; assert frames flow in and out.

â€¢ Record: `pip freeze > requirements.lock`; commit `requirements.lock` + install script.

4.3 Acceptance criteria (Codex verifies)

â€¢ All imports succeed inside ~/me3/.venv on the Pi (evidence: command output saved).

â€¢ A 10-second synthetic-audio pipeline runs without exception and logs frame counts.

â€¢ MCP SDK can spawn a trivial in-process server and list its tools (proves MCP path works).

â€¢ No secrets in any committed file; keys referenced by env var name only.

5 Task 2 â€” Build ASR & TTS MCP Servers (on Pi)

Owner: Antigravity Â· Auditor: Codex Â· These are OUR servers (built + run on the Pi), wrapping SOTA models. Commercial APIs are allowed for the LLM brain only â€” not for ASR/TTS.

5.1 ASR MCP server (faster-whisper 1.2.1)

â€¢ `pip install faster-whisper==1.2.1` â€” CTranslate2 Whisper, int8, on-Pi capable.

â€¢ Pick model size by RAM: Pi 4 (4GB) â†’ `small`/`base.en`; Pi 5 (8GB+) â†’ `medium`/`small` int8.

â€¢ Wrap with FastMCP: `from mcp.server.fastmcp import FastMCP; mcp = FastMCP("asr")`.

â€¢ Expose tools: `transcribe(wav_path_or_bytes) -> text`, `transcribe_stream(chunk) -> partial_text`.

â€¢ Serve over stdio (local) and/or SSE (network) so Pipecat can call it as an MCP tool.

â€¢ Test: feed a known WAV, assert exact expected transcript; measure RTF (real-time factor).

5.2 TTS MCP server (Piper 1.8.0)

â€¢ `pip install piper-tts==1.8.0` â€” low-latency neural TTS, real-time on Pi.

â€¢ Download a voice model (e.g. en_US lessac/amy low) into `~/me3/tts/voices/` (no key needed).

â€¢ Wrap with FastMCP: `FastMCP("tts")`; expose `synthesize(text) -> wav_bytes`.

â€¢ Stream chunks so Pipecat can start speaking before synthesis completes (latency win).

â€¢ Test: synthesize a sentence, assert non-empty WAV at 22050 Hz; measure chars/sec and RTF.

5.3 Acceptance criteria (Codex verifies)

â€¢ Both servers register and are discoverable by the Pipecat pipeline via MCP (tool list non-empty).

â€¢ ASR transcribes a 5 s sample correctly; RTF < 1.0 on target Pi (evidence: timing log).

â€¢ TTS emits valid audio; RTF < 1.0 (faster-than-realtime) on target Pi (evidence: timing log).

â€¢ Servers run as supervised processes (systemd) and restart on crash; logs captured.

6 Task 3 â€” VAD + Full-Duplex Engine

Owner: Antigravity Â· Auditor: Codex Â· Full-duplex = listen and speak at once, with barge-in. This is the hardest part; it is decomposed into verifiable microsteps.

6.1 VAD (Silero 6.2.3)

â€¢ `pip install silero-vad==6.2.3 torch` (CPU) â€” production VAD, endpointing built in.

â€¢ Load model; configure: sampling 16 kHz, `threshold=0.5`, `min_silence_ms=500`, `speech_pad_ms=300`.

â€¢ Feed 512-sample frames; emit `speech_start`/`speech_end` events with timestamps.

â€¢ Expose to pipeline as a frame processor (and optionally as an MCP tool for benchmarking).

â€¢ Tune on real room noise: sweep threshold, log false-trigger + miss rates on a 2-min sample.

6.2 Full-duplex loop (microsteps)

â€¢ Open a duplex stream: `sounddevice.RawInputStream(16k)` + `RawOutputStream(22050)` concurrently.

â€¢ Maintain a ring buffer of mic frames; VAD runs on it in a worker thread.

â€¢ On `speech_start` while TTS is playing â†’ CANCEL pending TTS queue (barge-in) and flush speaker.

â€¢ On `speech_end` â†’ slice the utterance, send to ASR, feed transcript to LLM brain.

â€¢ LLM reply â†’ TTS â†’ speaker, streamed in chunks; keep mic live throughout (no half-duplex gap).

â€¢ Echo/self-suppression: subtract or gate the agent's own output so it is not re-transcribed.

6.3 Acceptance criteria (Codex verifies)

â€¢ Agent speaks while mic stays open; a spoken interrupt stops the agent within ~300 ms (timed).

â€¢ Conversational exchange of â‰¥ 5 turns with no dead-air lockup and no self-transcription.

â€¢ VAD precision/recall logged on a fixed 2-min noisy sample; thresholds recorded in config.

â€¢ A 3-minute soak run produces no crash, leak, or buffer overflow (evidence: memory log).

7 Task 4 â€” Real-World Tasks + Deployment

Owner: Antigravity Â· Auditor: Codex Â· The agent must do useful things, not just talk. Tasks are exposed as MCP tools the LLM brain can invoke during conversation.

7.1 Real-world task tools (MCP)

â€¢ Weather: wrap a public/commercial weather API as an MCP tool `get_weather(city)` (key in env).

â€¢ Time/date: `now()`, `set_timer(seconds, label)` with a spoken confirmation + alarm tone.

â€¢ System control: `status()` (RAM/temp/free space), `play_tone(freq, ms)` for feedback.

â€¢ ME2 bridge (optional): call the existing ME2 voice-command model as a tool for offline intents.

â€¢ Each tool: typed schema, docstring, error handling, and a unit test with a mocked dependency.

7.2 Deployment & supervision

â€¢ systemd units: `me3-asr.service`, `me3-tts.service`, `me3-agent.service` (Restart=always).

â€¢ Boot persistence: enable all three; verify `systemctl status` after reboot.

â€¢ Config: single `config.yaml` (models, thresholds, transport, brain endpoint) â€” no hardcoded secrets.

â€¢ Logging: structured JSON logs to `~/me3/logs/`, rotated; include per-event timestamps for the benchmark.

â€¢ Safety: respect existing Pi folders (dandan VCM, Kiko); ME3 is self-contained in `~/me3`.

7.3 Acceptance criteria (Codex verifies)

â€¢ Agent completes â‰¥ 3 distinct real-world tasks by voice (e.g. 'what's the weather in Manila?', 'set a 5-minute timer', 'how much free space?') with correct tool output (evidence: logs).

â€¢ All services survive a reboot and self-heal on crash (evidence: journalctl excerpt).

â€¢ Tool-call trace shows the LLM selected the right MCP tool for each request (evidence: trace log).

8 Latency Benchmark (Classwork Deliverable)

A standardized, reproducible method to measure voice-interaction latency. Same fixtures, same clocks, same stats every run â€” so results are comparable across Pi 4 vs Pi 5 and across configs.

8.1 Metric definitions (fixed vocabulary)

â€¢ VAD latency: speech onset â†’ `speech_start` event.

â€¢ Endpoint latency: speech offset â†’ `speech_end` event (governed by min_silence_ms).

â€¢ ASR latency: utterance end â†’ final transcript (and first-partial latency separately).

â€¢ LLM TTFT: transcript ready â†’ first token of the reply.

â€¢ TTS TTFP: first reply token â†’ first audio chunk out of the speaker (time-to-first-playback).

â€¢ E2E voice-to-voice: user speech offset â†’ agent's first audible reply (the headline number).

8.2 Standardized method (the protocol)

â€¢ Fixtures: a frozen set of N=30 utterances (varied length/noise) stored as WAV; identical every run.

â€¢ Clocks: monotonic clock (`time.monotonic_ns`) stamped at each pipeline boundary; no wall-clock.

â€¢ Runs: 5 warm-up + 20 measured trials per fixture; report median, p95, and max (never mean alone).

â€¢ Isolation: single-task load, CPU governor `performance`, record `vcgencmd measure_temp` + `free -m`.

â€¢ Output: a JSON report `{fixture, metric, n, median_ms, p95_ms, max_ms, hw, cfg}` + a summary table.

8.3 Reporting & pass thresholds

â€¢ Deliver `benchmark.py` (runs the protocol) + `report.md` (table + interpretation) + raw JSON.

â€¢ Target (indicative, Pi 5 / int8): E2E voice-to-voice p95 < 1.5 s; TTS TTFP < 400 ms; ASR RTF < 1.0.

â€¢ Compare Pi 4 vs Pi 5 side-by-side in the same table; state which component dominates latency.

â€¢ Reproducibility gate: two consecutive runs differ < 10% on E2E p95, else investigate variance source.

9 Timeline, Risks & Handoff

ID	STEP	OWNER	GATE
S1	Pi pre-flight + venv + harness/MCP install	Antigravity	imports + smoke pipeline
S2	ASR MCP server (faster-whisper)	Antigravity	correct transcript, RTF<1
S3	TTS MCP server (Piper)	Antigravity	valid audio, RTF<1
S4	VAD (Silero) + tuning on room noise	Antigravity	precision/recall logged
S5	Full-duplex loop + barge-in	Antigravity	5-turn convo, ~300ms barge-in
S6	Real-world MCP task tools	Antigravity	3 tasks by voice
S7	systemd deploy + reboot persistence	Antigravity	survives reboot
S8	Latency benchmark run + report	Antigravity	JSON + report.md
S9	Independent (TTS-in-the-call) validation	Codex / OnIt	all gates green

9.2 Top risks & mitigations

â€¢ Pi 4 (4GB) too slow for medium Whisper â†’ mitigate: int8 + `small`/`base.en`; benchmark proves fit.

â€¢ Echo/self-transcription corrupts loop â†’ mitigate: AEC/gate + self-output suppression, verify in S5.

â€¢ Barge-in race (TTS not cancelling) â†’ mitigate: explicit cancel-on-speech-start, timed test < 300 ms.

â€¢ Commercial API cost/latency for LLM â†’ mitigate: swappable brain; local Ollama fallback for offline.

â€¢ Secrets leakage â†’ mitigate: env/keychain only; Codex greps diffs for key patterns before commit.

9.3 Handoff & completion definition

â€¢ Definition of Done: S1â€“S9 all 'verified' by Codex with evidence; benchmark report published.

â€¢ OnIt updates HANDOFF_INDEX.md (current state, open items, change log) + runs Update-HandoffIndex.ps1.

â€¢ AGENT_TASK_STATE.json set to status 'done'; next_step cleared; watchdog confirms 'no active task'.

â€¢ Left for user: physical Pi hardware access for live mic/speaker runs (agents cannot claim audio results without an executed, evidenced run).